

install-resource-pressure-timer

scripts/install-resource-pressure-timer.sh — hourly proactive host-pressure timer

Revision: 1 **Last modified:** 2026-08-18T19:40:00Z **Status:** active
Item: task #77 (BOB-076 2nd forced-logout incident follow-up)

Overview

`scripts/install-resource-pressure-timer.sh` installs and mechanically verifies a `systemd --user` timer that runs `challenges/scripts/resource_pressure_signature_challenge.sh` once an hour, independently of any build/commit activity. It is the standing between-builds sibling of `scripts/pre_build_verification.sh` invariant 25, which runs the same challenge but only when a build is actually invoked.

Why this exists

Commit 1f42357 landed `challenges/scripts/resource_pressure_signature_challenge.sh` — a 5-signature detector of the conditions that preceded the project's 2nd forced-logout incident (2026-08-18, see `docs/incidents/2026-08-18-perceived-forced-logout-2nd.md`). The incident's own §12.12 triage found a container EAGAIN cascade roughly 5 minutes *before* `user@1000.service` was SIGKILLED — a leading indicator that a standing, regularly-scheduled check would have caught well before the kill, if one had existed. Wiring the challenge only into the pre-build gate (invariant 25) closes the “at build time” gap but leaves the “between builds” gap open — most of any given day has no build running. This script closes that second gap.

Prerequisites

- Linux host with a systemd --user session available (systemctl --user status must succeed). Confirmed via `_require_linux_systemd` at the top of every invocation — a non-Linux host or an unavailable user session is an honest, actionable exit 1, never a silent no-op.
- `loginctl show-user "$(id -un)" -p Linger = yes` is recommended (not required by this script) so the timer also fires when the operator is logged out, matching the existing `boba-svc.sh / scripts/install.sh` guidance for the rest of the boba stack.

Usage

```
bash scripts/install-resource-pressure-timer.sh          #  
    symlink install (default)  
bash scripts/install-resource-pressure-timer.sh --copy    #  
    hard-copy install  
bash scripts/install-resource-pressure-timer.sh --uninstall #  
    stop + disable + remove
```

Symlink mode (default) means a future `git pull` that updates `scripts/systemd/user/boba-resource-pressure-check.{service,timer}` is picked up automatically after the next `systemctl --user daemon-reload` — no re-install needed. `--copy` hard-copies instead, matching `scripts/boba-svc.sh install --copy`'s convention for the rest of the stack's units.

The script is **idempotent** — re-running it re-symlinks (or re-copies), reloads, re-enables, and re-verifies; it never errors on an already-installed timer.

What it installs

Two unit files, source-of-truth at `scripts/systemd/user/boba-resource-pressure-check.{service,timer}` (same directory convention as the existing `boba.target / boba-stack.service / boba-webui-bridge.service` units — see `scripts/boba-svc.sh`), installed into `~/.config/systemd/user/`:

- **boba-resource-pressure-check.service** — `Type=oneshot`, runs `challenges/scripts/resource_pressure_signature_challenge.sh` directly. Deliberately **not** `PartOf=boba.target` — resource-pressure monitoring must run whether the qBittorrent/Jackett stack is up or down.
- **boba-resource-pressure-check.timer** — `OnBootSec=5min`, `OnUnitActiveSec=1hour`, `RandomizedDelaySec=5min`, `Persistent=true` (a missed fire — e.g. host was suspended or the user was

logged out — runs on next login instead of being silently skipped).

Mechanical verification (§11.4.6 — never assume)

The install script does **not** consider itself done just because enable + start returned exit 0. It performs two independent, captured checks:

1. **The timer is actually scheduled.** `systemctl --user list-timers --all` output is captured and grepped for `boba-resource-pressure-check.timer`. Absence is a hard exit 1 naming the exact evidence file to inspect.
2. **An immediate first fire actually executes.** `systemctl --user start --wait boba-resource-pressure-check.service` is run synchronously (this executes the real challenge script once, on the spot), then `systemctl --user status + journalctl --user -u ... -n 50` are captured. The script then reads `systemctl --user show ... -p Result --value` and reports **two separate, honest facts** rather than conflating them:
 - Did the unit *resolve and execute at all* (the wiring)? This is verified via `systemctl --user cat` succeeding — independent of the challenge's own exit code.
 - Did the challenge itself find real host pressure *right now* (Result=success vs Result=exit-code)? This is real, separate information about host state, not an installation defect, and is surfaced as a WARN with the diagnostic path, never silently swallowed and never treated as an install failure.

Evidence

Every run writes to `docs/qa/task-77/` — deliberately `.txt`, not `.log`: the repo's `.gitignore` has a blanket `*.log` rule (verified — `docs/qa/B0B-076/*.log` is gitignored despite living under the tracked `docs/qa/` tree), so a `.log` extension here would silently produce untracked evidence. `.txt` matches the existing `docs/qa/B0B-075/*.txt` precedent for evidence meant to be committed (§11.4.83). Contrast with invariant 25's own evidence directory (`docs/qa/pre_build_resource_pressure/`), which deliberately DOES use `.log` because that evidence is a per-pre-build-run artifact meant to stay local-only, not a one-time install record:

File	Contents
<code>install_output.txt</code>	Full stdout+stderr of the entire install run (via <code>exec >>(tee -a ...) 2>&1</code> , flushed

File	Contents
	before exit — see “Implementation note” below).
list_timers.txt	Raw systemctl --user list-timers --all output at verification time.
first_fire_status.txt	The immediate first fire’s systemctl --user status + journalctl output.

Exit codes

Code	Meaning
0	Units installed, timer scheduled (verified), first fire executed (verified) — regardless of whether the challenge itself found pressure.
1	Installation-level failure: missing unit source, list-timers verification failed, or the service does not resolve via systemctl --user cat (daemon-reload / symlink-target problem). Also returned by --uninstall’s own platform guard.
2	Invocation error (unknown flag).

Implementation note — output capture without dropped stderr

An early draft piped each `_info/_warn/_error` call individually through a `_tee()` helper (`msg | _tee`). This silently dropped every `_error` line from the evidence log, because `_error` writes to `stderr` and a bare pipe (`|`) only captures a command’s `stdout` — the error text still printed live to the terminal (bypassing the pipe entirely) but never reached `install_output.txt`. Caught during this task’s own review pass before landing; fixed by switching to a single `exec > >(tee -a "${INSTALL_LOG}") 2>&1` redirection for the whole script body, plus an `EXIT` trap that closes the script’s own `stdout/stderr` and waits on the `tee` subprocess so the log can never be silently truncated by the script exiting before `tee` finishes flushing.

Related scripts

- challenges/scripts/resource_pressure_signature_challenge.sh — the 5-signature detector this timer runs.
- scripts/pre_build_verification.sh invariant 25 (CM-RESOURCE-PRESSURE-SIGNATURE-CHECK) — the build-time, non-blocking sibling coverage for the same challenge.
- scripts/boba-svc.sh / scripts/systemd/user/ — the existing systemd --user unit convention this script follows.
- docs/incidents/2026-08-18-perceived-forced-logout-2nd.md — the incident this whole mechanism exists to catch earlier next time.

Uninstalling

```
bash scripts/install-resource-pressure-timer.sh --uninstall
```

Stops the timer, disables it, removes both installed unit files from ~/.config/systemd/user/, and reloads the user systemd manager. The in-repo unit sources under scripts/systemd/user/ are left untouched. Note: because the default install mode places a **direct symlink** into ~/.config/systemd/user/ (not a copy), systemd treats it as a “linked” unit, and systemctl --user disable on a linked unit removes the link itself (not merely an [Install] .wants/ symlink) — so the uninstall loop’s per-file existence check may legitimately find the timer’s symlink already gone by the time it runs. This is expected systemd behavior, not a bug (verified empirically during task #77’s own testing).